

 Reusable def Functions

INSTANT

 LEGB Scope & Global Data Analysis Diploma • Session 12

PYTHON

FUNCTIONS & STRINGS

Functions · Parameters · LEGB Scope · Lambda · Nested Loops & String
Methods

4 Core Topics

Curriculum Modules

Functions • Scope • Lambda •
Strings

20 Slides

Final Python Session

Python Fundamentals
Capstone

Live Sandbox

Interactive Code Labs

Password, Palindrome & Cart
Demos

[Start Session 12 →](#) [Lambda & Map/Filter](#)[T String Methods & Loops](#)

— WHAT WE WILL COVER

Master Functions, Scope & Text Processing

Everything you need to write modular, clean, and production-ready Python code:

01



FUNCTIONS & PARAMETERS

DRY principle, def syntax, positional vs keyword args, default values, *args & **kwargs.

02



RETURN VALUES & SCOPE

Returning single or multiple values as tuples, early exits, and the LEGB variable scope rule.

03



LAMBDA & FUNCTIONAL TOOLS

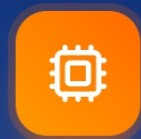
Anonymous one-line functions, map(), filter(), and custom sorting with key functions.

04



LOOPS & STRING METHODS

Nested loop matrix traversal, string methods, slicing [::-1], and real-world capstone mini-projects.



PART 01

FUNCTIONS & PARAMETERS

Building Blocks of Clean, Reusable Code

Learn to encapsulate logic into modular blocks and make functions flexible with parameter options.

— UNDERSTANDING FUNCTIONS

What Are Functions & Why Use Them?

i Definition: A function is a named, reusable block of code that performs a specific task. It accepts inputs, processes them, and returns an output.

WHY

Organize complex code into manageable pieces, reuse code without copy-pasting, and adhere to the DRY Principle (Don't Repeat Yourself).

Analogy: A cooking recipe — write it once, execute it whenever you need that dish!

HOW

Define with the `def` keyword followed by function name and parentheses `()`. Call by name with arguments.

KEY CHARACTERISTICS

</> PYTHON CODE EXAMPLE

Python 3.x

```
# Defining a simple function
def greet(name):
    return f"Hello, {name}!"

# Calling the function
message = greet("Alice")
print(message) # Output: Hello, Alice!
```

— POSITIONAL VS KEYWORD

Positional and Keyword Arguments

Controlling how arguments are passed into function parameters:

POSITIONAL ARGUMENTS

Passed in the exact order parameters are defined. Swapping order changes output or causes bugs!

```
greet("Alice", "Hello") # Hello, Alice!
```

KEYWORD ARGUMENTS

Passed by naming parameters explicitly (**name=value**). Order does not matter!

```
greet(greeting="Hi", name="Bob") # Hi, Bob!
```

i Mixing Rule: Positional arguments MUST come before keyword arguments!

</> FUNCTION CALL EXPLORER

```
def greet(name, greeting):
```

Select Call Type to Test:

1. Positional

2. Keyword

3. Mixed (Valid)

4. Mixed (Invalid ❌)

Function Call:

```
greet("Alice", "Hello")
```

Output: "Hello, Alice!"

— DEFAULT VALUES, *ARGS & **KWARGS

Default & Variable-Length Arguments

Handling preset parameters and an arbitrary number of positional or keyword inputs:

*ARGS (POSITIONAL TUPLE)

Collects any number of positional arguments into a **Tuple**.

```
def total(*numbers):
    return sum(numbers)
```

Current *args:

total(10, 20, 15)

Result:

45

+ Add Random Number to *args

**KWARGS (KEYWORD DICT)

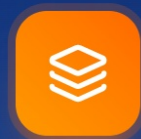
Collects any number of keyword arguments into a **Dictionary**.

```
def profile(**info):
    for k, v in info.items():
        print(f"{k}: {v}")
```

Rendered **kwargs dict:

```
profile(name="Ali", age=25, role="Analyst")
```

```
→ info = { "name": "Ali", "age": 25, "role": "Analyst" }
```



PART 02

RETURN VALUES & VARIABLE SCOPE

Getting Data Back & Controlling Variable Access

Master return statements, tuple unpacking from functions, and the LEGB variable lookup hierarchy.

THE RETURN STATEMENT

The Return Statement — Sending Data Back

Definition: The return statement exits a function immediately and sends a result back to the caller.

WHY

Without return, a function returns None by default. return lets you calculate values inside functions and use them elsewhere in your program.

Note: You can return multiple values separated by commas — Python packages them into a tuple!

HOW

Write return followed by the variable or expression. Any code after return in the function block is unreachable.

KEY CHARACTERISTICS

- Exits Immediately
- Returns Data to Caller

PYTHON CODE EXAMPLE

Python 3.x

```
# Single return value
import math

def circle_area(radius):
    return math.pi * (radius ** 2)

area = circle_area(5)
print(f"Area: {area:.2f}") # 78.54

# Multiple return values (Tuple)
def min_max(numbers):
    return min(numbers), max(numbers)
```

— THE LEGB RULE

Understanding Scope — The **LEGB** Rule

Where variables live and how Python searches for them in order:

L - Local

E - Enclosing

G - Global

B - Built-in

SCOPE LAYER: LOCAL

Inside the current function. Created when called, destroyed when function returns.

💡 **Lookup Order:** Python checks **Local** → **Enclosing** → **Global** → **Built-in**. If not found in any, raises **NameError**!

MODIFIED GLOBAL VARIABLE DEMO

count = 100

```
count = 100 # Global (G)

def update_count():
    global count # Declare intent to modify
    count = 200 # Modifies global variable
```

⚡ **Execute update_count() (+100)**

Log: count = 100 (Global)



PART 03

LAMBDA & FUNCTIONAL TOOLS

Anonymous Functions, map() & filter()

Write quick inline operations and transform or filter collections elegantly.

LAMBDA FUNCTIONS

Lambda Functions — Compact One-Liners

Definition: A lambda function is a small, anonymous function defined with the lambda keyword. It can take any number of arguments, but can only have one expression.

WHY

Use lambda when you need a short, temporary function for a brief operation — especially as arguments to functions like `sorted()`, `map()`, or `filter()`.

Rule of thumb: If it requires multiple lines or complex logic, use standard `def`!

HOW

Syntax: `lambda arguments: expression` (the result is returned automatically).

KEY CHARACTERISTICS

PYTHON CODE EXAMPLE

Python 3.x

```
# Regular function vs Lambda
def add_regular(x, y):
    return x + y

# Lambda equivalent
add_lambda = lambda x, y: x + y
print(add_lambda(5, 3)) # 8

# Quick utility lambdas
square = lambda x: x ** 2
is_even = lambda x: x % 2 == 0
greet = lambda name: f"Hello, {name}!"
```

— MAP() & FILTER()

map() and filter() Functions

Transforming and selecting elements in iterables without explicit for loops:

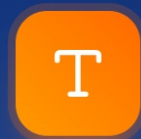
1. `map(lambda x: x**2, numbers)`

2. `filter(lambda x: x > 0, numbers)`

```
numbers = [ 1 -2 3 -4 5 ]
```

→ Applies `lambda x: x**2` to EVERY item

```
list(map(...)) = [ 1 4 9 16 25 ]
```



PART 04

LOOPS & STRING METHODS

Iteration Control & Powerful Text Processing

Combine nested loops with string manipulation methods for data parsing and validation.

FOR & NESTED LOOPS

For Loops & Nested Loops in Depth

Multi-dimensional iteration, matrix traversal, and iteration step mechanics:

NESTED LOOPS MECHANICS

For every single iteration of the outer loop, the inner loop completes **ALL** of its iterations!

```
for row in range(3): # Outer (0,1,2)
    for col in range(3): # Inner (0,1,2)
        print(f"({row}, {col})")
```

⚠ Performance Note: 2 nested loops of size N take $N \times N$ iterations ($O(N^2)$). Keep nesting shallow!

3x3 MATRIX TRAVERSAL

Active: (0, 0)

(0,0)	(0,1)	(0,2)
(1,0)	(1,1)	(1,2)
(2,0)	(2,1)	(2,2)

▶ Step Next Iteration →

— STRING METHODS TOOLKIT

Python String Methods Toolkit

Essential methods for cleaning, transforming, splitting, and validating text data:

TEST INPUT STRING:

Length: 25 chars

Hello Python Analyst!

`.lower()`

`.upper()`

`.strip()`

`.replace()`

`.split(" ")`

`[::-1]`

Action Result:

`hello python analyst!`

— SLICING & PALINDROME CHECKER

String Slicing & Palindrome Checker

Definition: String slicing allows extracting substrings using `text[start:stop:step]`. Reversing a string is achieved with `text[::-1]`.

WHY

String slicing combined with string cleaning (`.lower()`, `.replace()`) allows creating elegant text verification algorithms like Palindrome Checkers.

Analogy: Reading a word forward and backward — e.g., "racecar", "madam".

HOW

Clean the input by lowercasing and removing spaces, then compare the cleaned string with its reverse `[::-1]`.

KEY CHARACTERISTICS

✓ Immutable Strings

✓ Step -1 Reverses

</> PYTHON CODE EXAMPLE

Python 3.x

```
# Palindrome Checker Function
def is_palindrome(text):
    # 1. Clean string: lower case & remove spaces
    cleaned = text.lower().replace(" ", "")
    # 2. Compare with reversed version
    return cleaned == cleaned[::-1]

# Testing Palindromes
print(is_palindrome("Racecar"))           # True
print(is_palindrome("A man a plan a canal Panama")) # True
```

— 4 REAL-WORLD EXAMPLES

Real-World Function Projects

Four complete practical examples combining functions, parameters, string methods, and loops:

Temperature Converter

```
def celsius_to_f(c):
    return (c * 9/5) + 32
```

celsius_to_f(25) → 77.0 °F

Shopping Cart Total

```
def cart_total(*prices, tax=0.05):
    return sum(prices) * (1 + tax)
```

cart_total(10, 20, 15, tax=0.08) → \$48.60

Palindrome Checker

```
cleaned = text.lower().replace(" ", "")
return cleaned == cleaned[::-1]
```

is_palindrome("Racecar") → True

Password Strength Checker

```
has_digit = any(c.isdigit() for c in pwd)
return has_digit and len(pwd) >= 8
```

check_password("Admin123") → True

— PYTHON FUNCTION SIMULATOR

Interactive Function Execution Lab

Test temperature conversion, cart total, palindrome checking, and password strength live:

>_ LIVE FUNCTION SANDBOX
Interactive Python Interpreter

Password Strength Checker Lab:

Admin123

len >= 8: ✓ Pass

has digit: ✓ Pass

has upper: ✓ Pass

check_password("Admin123"):

True (STRONG 🔒)

— COMPLETE RECAP

Python Fundamentals — Master Summary

Summary of everything learned across Sessions 09, 10, 11, and 12:

SESSION	TOPIC AREA	CORE CONCEPTS & KEYWORDS	PRIMARY USE CASE
Session 09	Python Fundamentals	variables, int, float, str, type casting, operators	Basic arithmetic, data store, type conversions
Session 10	Control Flow & Loops	if, elif, else, while, break, continue, pass	Decision making & condition-driven loops
Session 11	Data Structures	lists, tuples, dicts, sets, slicing, comprehensions	Grouping collections, key-value lookup, unique sets
Session 12	Functions & Strings	def, return, LEGB scope, lambda, map/filter, nested loops, str methods	Modular reusable code, scope control & text parsing

INSTANT

✓ SESSION 12 COMPLETED SUCCESSFULLY

THANK YOU

Thank you for your active participation!

👉 **Module 02 Complete! Next Module: 03 Python Libraries – NumPy Arrays, Pandas DataFrames & Data Visualizations**

↺ Restart Presentation